

个人实训报告：Baseline 评测、Sort 优化与 Rope 阶段化探索

赛题 9 第 2 组：多机器人协同的具身操作

26220478 沈鼎丁

1. 个人分工概述

本次实训中，我的主要工作可以分为三个部分：

- Baseline 评测与问题定位**：第一天主要负责环境配置、模型接入、六个任务 baseline 运行和失败类型整理，为小组后续优化提供对照数据。
- Sort Cubes 主线优化**：围绕 Sort 任务的 plan 模式下做较完整的任务级优化，包括合法动作枚举、handoff 规则、verifier、失败动作精确屏蔽、partial fallback、IK 与 holding recovery 等。
- Move Rope 早期探索**：对 Rope 任务进行早期约束修复和阶段化分析，包括非法动作修复、prompt 污染修复、WAIT/PATH 统一，以及“双 PICK -> 双 PUT”阶段策略设计。

此外，第二天小组成员提出了更统一的新架构后，我也参与了 **Sort 方法与小组统一 arbitration / verifier 思路的讨论和融合**，使我在单任务中验证有效的策略能够接入小组最终框架。

2. Baseline 评测与问题定位

实训初期，在阅读 RoCoBench 论文后，我首先完成了环境、模型和评测链路的跑通。由于项目包含 6 个多机器人具身操作任务：

```
sort, cabinet, rope, sweep, sandwich, pack
```

如果直接开始修改某一个任务，很容易只凭直觉调 prompt，难以判断真正瓶颈。因此我先跑了六任务 baseline，并检查每个 run 中的 `steps*_success_*.json`、prompt、feedback、parser 输出和执行日志。

2.1 Baseline 工作内容

我的 baseline 工作主要包括：

- 配置运行环境、MuJoCo/EGL、模型服务和数据输出目录；
- 使用 `run_dialog.py` / `evaluator.py` 跑通不同任务；
- 对 plan / dialog 两种通信模式做对照；
- 记录成功率、失败 step 和失败类型；

2.2 失败类型归纳

通过 baseline，我发现失败并不是单一的“prompt 不好”，而是多类问题叠加：

失败类型	表现	影响
Parser / 格式错误	输出动作名不存在、缺少 PATH、格式不符合 parser	计划无法进入执行

失败类型	表现	影响
IK failed	高层动作看似合理，但机械臂到不了目标位姿	任务卡死或反复 replan
Collision	多机器人同时进入共享区域或路径过近	执行失败，后续状态变差
Semantic invalid	动作违反任务规则，如顺序错误、单端放 Rope	即使能解析也不应执行
Timeout / 空转	多轮失败后没有有效进展	run 到 step 上限仍失败

这些观察帮助小组明确：多机器人身任务不能只依赖 LLM 自由规划，还需要结合任务规则、可达性、IK、碰撞和反馈恢复机制。

2.3 Baseline 结果与优化方向选择

仓库中可追溯的部分 baseline 结果如下：

模式	Sort Cubes	Arrange Cabinet	Move Rope	Sweep Floor	Make Sandwich	Pack Grocery
Plan	0/5	2/5	1/5	0/5	1/5	0/5
Dialog	1/5	1/5	0/5	2/5	0/5	0/5

其中观察到的部分问题如下：

- cabinet：需要更强阶段规划和物体放置策略
- sweep：工具、垃圾和 SWEEP 阶段容易不匹配
- sandwich：容易出现食材顺序错误或重复动作。dialog 有成功样本，但稳定性不足
- pack：PATH、放置区域、IK 和碰撞问题明显
- rope：绳子自身的柔软特性导致 IK 和碰撞问题明显，路径不合法
- sort：机械臂会把物体放到另一个机械臂操作范围之外，导致后期无法够取，动作失效

这是论文使用 GPT-4 测的 benchmark：

		Pack Grocery	Arrange Cabinet	Sweep Floor	Make Sandwich	Sort Cubes	Move Rope
Central Plan (oracle)	Success step, replan	0.82 ± 0.06 11.1, 3.9	0.90 ± 0.07 4.0, 2.7	1.00 ± 0.00 8.4, 2.0	0.96 ± 0.04 8.8, 1.2	0.70 ± 0.10 8.6, 2.6	0.50 ± 0.11 2.3, 3.9
Dialog w/o History	Success step, replan	0.48 ± 0.11 9.2, 3.1	1.00 ± 0.00 4.2, 1.4	0.00 ± 0.00 N/A, 1.0	0.33 ± 0.12 9.6, 1.8	0.73 ± 0.11 5.8, 1.4	0.65 ± 0.11 3.7, 3.1
Dialog w/o Feedback	Success step, replan	0.35 ± 0.10 18.0, 1.0	0.70 ± 0.10 5.9, 1.0	0.95 ± 0.05 7.6, 1.0	0.35 ± 0.11 12.6, 1.0	0.53 ± 0.13 4.9, 1.0	0.45 ± 0.11 3.4, 1.0
Dialog (ours)	Success step, replan	0.44 ± 0.06 9.9, 3.5	0.75 ± 0.10 4.7, 2.0	0.95 ± 0.05 7.1, 1.0	0.80 ± 0.08 10.2, 1.7	0.93 ± 0.06 4.9, 1.3	0.65 ± 0.11 2.5, 3.1

Table 2: Evaluation results on RoCoBench. We report averaged success rates $\uparrow$ over 20 runs per task, the average number of steps in *successful* runs $\downarrow$, and average number of re-plan attempts used across all runs $\downarrow$.

在 RoCoBench 论文中，在 central plan 模式下，即使使用 GPT-4，Sort 和 Rope 这类强协作任务的成功率仍然较低，大约只有 50%–70%。根据这些结果，我选择将主要精力用中心规划路径投入到 **Sort** 和 **Rope**。Sort 规则离散、适合验证 verifier / fallback 思路；Rope 难度高，适合分析 LLM 在路径、阶段协同和柔性物体任务中的局限。

3. 主要成果：Sort Cubes 任务优化

3.1 任务难点

Sort Cubes 要求 Alice、Bob、Chad 三个机器人将不同 cube 放到对应 panel：

```
blue_square -> panel2
pink_polygon -> panel4
yellow_trapezoid -> panel6
```

三个机器人可达范围不同，跨区域移动必须通过共享 handoff panel 完成：

```
Alice / Bob 通过 panel3 交接
Bob / Chad 通过 panel5 交接
```

因此，Sort 的难点不是“知道目标 panel”，而是：每一步动作必须符合当前状态、机器人可达范围、交接方向、IK 和碰撞约束。

3.2 初始失败现象

在 baseline 和早期实验中，Sort 主要失败包括：

- LLM 输出格式正确但物理不可执行的动作；
- cube 被放到后续机器人无法接手的位置；
- Bob/Chad 等机器人在共享区域并行动作时碰撞；
- 某个机器人 IK failed 后，模型反复输出同一失败动作；
- 执行中出现机器人已经 holding cube，但下一轮仍被推荐 PICK。

这些问题说明，仅靠自然语言 prompt 很难让 LLM 稳定遵守所有物理约束。

3.3 解决方案一：Legal Actions 与 Handoff 规则

我首先为 Sort 加入状态驱动的合法动作集合。系统根据当前环境状态判断：

- 每个 cube 当前在哪个 panel；
- 是否已经到达目标；
- 哪个机器人能抓到它；
- 它下一步应该沿哪个方向 handoff；
- 目标 panel 是否适合当前机器人放置。

这样 LLM 不再完全自由生成动作，而是在当前状态下的合法动作中选择。

同时，我加入了方向性 handoff 规则。例如跨区域转运必须通过 panel3 或 panel5，避免 cube 被放到后续无人可达的位置。这一部分使 Sort 从“自由规划”变成了“受任务规则约束的规划”。

3.4 解决方案二：Verifier 与 Failure-specific Ban

在 `plan_prompter` 层，我加入了 LLM 输出前置校验，检查：

- 是否包含 `EXECUTE`；
- 是否每个机器人 `exactly one action`；
- `action` 是否属于当前 `legal actions`；
- 是否多个机器人操作同一个 `cube`；
- 是否多个机器人放到同一个目标；
- 是否重复之前失败的动作。

早期失败恢复有一个问题：如果 Bob 的动作 `IK failed`，旧逻辑可能把 Alice 和 Chad 的动作也一起禁掉，导致系统无法执行本来可行的部分动作。因此我改成从环境反馈中识别失败机器人，例如：

```
IK failed: on Bob
Out of reach: Bob
```

然后只 ban 失败机器人的当前动作，保留其他机器人的可执行动作。这提升了系统的恢复能力。

3.5 解决方案三：Partial Fallback、IK 与 Holding Recovery

当 LLM 多次 `replan` 失败后，我加入 `deterministic partial fallback`：

1. 优先尝试任务推荐计划；
2. 如果整体计划失败，尝试推荐计划的子集；
3. 如果双机器人组合失败，退化到单机器人动作；
4. 避免一个机器人失败拖垮整轮执行。

此外，我还针对 Sort 的物理执行问题做了补充修复：

- 调整 `pane13` / `pane15` 的共享 `handoff` 点，使“放的人能放，接的人也能抓”；
- 对 Sort `PICK` 姿态做放松，减少 `cube yaw` 或 `handoff` 后姿态变化导致的 `IK failed`；
- 增加轻量 `IK precheck`，提前过滤明显不可行的 `PICK`；
- 增加 `PLACE <object> <pane1>` 形式的 `PLACE-only recovery`，处理机器人已经 `holding cube` 的状态；
- 尝试 `safe parallel`，只允许低风险并行，避免 Bob 与其他机器人在共享区域碰撞。

3.6 Sort 实验结果

实验版本	结果	说明
<code>sort_legal_plan_test</code>	4/7 成功	初步加入 <code>legal action</code> 后仍有碰撞和卡死
<code>sort_verifier_test</code>	10/10 成功	串行 <code>verifier</code> 版本稳定性最高
<code>sort_safe_parallel_test</code>	7/10 成功	并行提高效率，但稳定性下降

实验版本	结果	说明
<code>sort_safe_parallel_recovery_test</code>	9/10 成功	加入 recovery 和 IK 改进后稳定性提升

Sort 的优化说明：对于规则相对离散的多机器人任务，LLM 可以负责高层选择，但必须通过 legal actions、verifier、fallback 和 recovery 机制保证物理可执行性。

4. 拓展工作：Move Rope 早期约束与阶段化探索

4.1 Rope 任务难点

Rope 任务要求 Alice 和 Bob 分别抓住绳子两端，将绳子越过障碍墙并放入窄槽。相比 Sort，Rope 更难，因为它不仅需要选择动作，还需要输出 PATH，并且 rope 是柔性物体，单端操作会严重破坏后续状态。

4.2 早期问题与修复

我在 Rope 早期探索中主要解决了几类基础问题。

第一，模型经常输出不存在的动作名，例如：

```
LIFT rope_front_end
PLACE rope_front_end groove_left_end
```

但 parser 实际支持的是 `PICK`、`PUT` 和 `WAIT`。因此我修正了 Rope action prompt，明确禁止 `LIFT` / `PLACE` / `MOVE` 等非法动作，并将误导性示例改为合法的 `PUT`。

第二，Rope prompt 曾混入 Sort 的 `cube / panel` 规则，导致任务描述污染。我参与修复了 prompt 生成逻辑，使不同任务优先使用自己的 `central_plan_prompt()`。

第三，Rope 使用 `action_and_path` 模式，所有动作都需要 PATH。原来 prompt 允许 `WAIT`，但 feedback 又把 `WAIT` 判非法，前后矛盾。我将 `WAIT` 统一为合法动作，并要求 `WAIT` 也带四个路径点。

第四，Rope `PICK` 目标高度太低时容易撞桌或 IK failed。我将 rope 端点的 `PICK` 高度改为带上下界的安全高度，减少贴桌面抓取的问题。

4.3 阶段化策略：双 PICK -> 双 PUT

Rope 的核心问题是不能单端操作。因此我提出并实现了早期阶段化策略：

```
阶段 1: Alice 和 Bob 先分别 PICK rope 两端
阶段 2: 只有两端都被抓住后，才允许两人同时 PUT
如果只有一个机器人 holding, holding robot WAIT, 另一个机器人先 PICK
禁止 PICK 和 PUT 混在同一轮
禁止单机器人 PUT
```

这套策略被写入 prompt、recommended plan 和 feedback 检查中，用于减少 LLM 自由规划导致的 rope 状态破坏。

4.4 局限与后续工作

Rope 早期探索虽然减少了非法动作和阶段错误，但仍然存在明显局限：

- IK 对 rope end 抓取位姿非常敏感；
- PATH 点稍有偏差就可能导致碰撞或不可达；
- 柔性物体动力学会放大单步错误；
- dialog baseline 0/5，说明多轮对话不能自动解决底层 IK 和路径可执行性问题。

因此，Rope 的最终优化不能只靠 prompt 或对话协商，还需要更强的路径生成、IK 友好的姿态约束和任务级 deterministic fallback。后续小组成员在此基础上继续优化 Rope，取得了比早期探索更好的最终效果。我的贡献主要是完成早期失败定位、动作空间修复和阶段化分析，为后续优化提供基础。

5. 典型实验记录

5.1 Sort 成功案例：串行 verifier 版本稳定完成

- 路径：`data/sort_verifier_test/run_0`
- 结果文件：`data/sort_verifier_test/run_0/steps5_success_True.json`
- 结果摘要：`success=true`，结束于 step 5，用时约 91.45s。

该 run 中，系统基本采用保守串行动作，每一轮只让关键机器人执行一个可控动作，避免共享区域碰撞。部分执行序列如下：

```
step_0: Alice WAIT | Bob PICK yellow_trapezoid PLACE panel5 | Chad WAIT
step_1: Alice WAIT | Bob WAIT | Chad PICK yellow_trapezoid PLACE panel6
step_2: Alice WAIT | Bob PICK blue_square PLACE panel3 | Chad WAIT
step_3: Alice PICK blue_square PLACE panel2 | Bob WAIT | Chad WAIT
step_4: Alice PICK pink_polygon PLACE panel3 | Bob WAIT | Chad WAIT
step_5: Alice WAIT | Bob PICK pink_polygon PLACE panel4 | Chad WAIT
```

这组记录说明：在 Sort 这种离散 handoff 任务中，牺牲一部分并行效率、优先保证动作合法性和可执行性，可以显著提高稳定性。

5.2 Sort 失败案例：并行动作导致 Bob/Chad 碰撞

- 路径：`data/sort_legal_plan_test/run_3`
- 结果文件：`data/sort_legal_plan_test/run_3/steps9_success_False.json`
- 结果摘要：`success=false`，结束于 step 9，用时约 220.32s。

典型失败发生在并行动作中：

```
Previous Plan:
NAME Alice ACTION WAIT
NAME Bob ACTION PICK blue_square PLACE panel3
NAME Chad ACTION PICK yellow_trapezoid PLACE panel6

Environment Feedback:
Collision detected: collided object pairs: Chad-Bob
```

这个案例说明，只从符号层面看，Bob 搬 `blue_square`、Chad 搬 `yellow_trapezoid` 都是合理动作；但两个机器人同时进入中间共享区域时，会产生碰撞风险。因此后续我加入 `safe parallel` 约束，只允许低风险并行，并在碰撞反馈后避免把单个动作过度 ban 掉。

5.3 Sort 改进案例：safe parallel + recovery 后恢复稳定性

- 路径： `data/sort_safe_parallel_recovery_test/run_0`
- 结果文件： `data/sort_safe_parallel_recovery_test/run_0/steps5_success_True.json`
- 结果摘要： `success=true`，结束于 step 5，用时约 103.61s。

该 run 中，系统允许 Alice 和 Chad 在低风险情况下并行，同时保留 Bob 的中间区域保守策略。部分动作如下：

```
step_0: Alice PICK yellow_trapezoid PLACE panel3 | Bob WAIT | Chad PICK
blue_square PLACE panel5
step_1: Alice WAIT | Bob PICK yellow_trapezoid PLACE panel5 | Chad WAIT
step_2: Alice PICK pink_polygon PLACE panel3 | Bob WAIT | Chad PICK
yellow_trapezoid PLACE panel6
step_3: Alice WAIT | Bob PICK pink_polygon PLACE panel4 | Chad WAIT
step_4: Alice WAIT | Bob PICK blue_square PLACE panel3 | Chad WAIT
step_5: Alice PICK blue_square PLACE panel2 | Bob WAIT | Chad WAIT
```

这组记录体现了后续策略的折中：不是完全禁止并行，而是只开放安全并行，并配合 `verifier / recovery` 保持可执行性。

5.4 Rope 成功案例：双 PICK 后双 PUT

- 路径： `data/rope_prompt_fix/run_0`
- 结果文件： `data/rope_prompt_fix/run_0/steps1_success_True.json`
- 结果摘要： `success=true`，结束于 step 1，用时约 85.18s。

该 run 的动作非常符合我提出的阶段化策略：

```
step_0:
Alice PICK rope_front_end PATH [...]
Bob   PICK rope_back_end PATH [...]

step_1:
Alice PUT rope_front_end groove_left_end PATH [...]
Bob   PUT rope_back_end groove_right_end PATH [...]
```

该案例说明，对 Rope 这类柔性物体任务，关键不是让单个机器人自由尝试，而是先保证两端同时被抓住，再进行双臂协同放置。

5.5 Rope 失败案例：IK 与桌面碰撞问题

- 路径： `data/rope_prompt_fix/run_1`
- 结果文件： `data/rope_prompt_fix/run_1/steps2_success_False.json`
- 结果摘要： `success=false`，`timed_out=true`，用时约 66.85s。

典型反馈如下：

```
Previous Plan:
NAME Alice ACTION PICK rope_front_end PATH [...]
NAME Bob ACTION PICK rope_back_end PATH [...]

Environment Feedback:
IK failed: on Bob (-0.55, 0.31, 0.55)
```

另一次反馈中还出现：

```
Collision detected: collided object pairs: Alice-table
IK failed: on Bob (-0.55, 0.31, 0.52)
```

这个案例说明，Rope 的核心瓶颈不只是动作顺序，还包括底层 PATH 与 IK 可执行性。即使 prompt 中写清楚了“双 PICK -> 双 PUT”，如果路径点或抓取位姿不适合机器人，仍然会失败。这也是我后续把 Rope 定位为“需要路径生成与 IK 友好约束继续优化”的原因。

6. 与小组协作和架构融合

第一天，我主要在自己的代码分支中完成环境配置、baseline 评测、Sort/Rope 失败定位，以及 Sort 任务级 verifier / fallback 的初步实现。与此同时，组员提出了一个更统一的新架构，希望用更通用的 arbitration / verifier 机制管理多任务规划。

第二天，我将自己在 Sort 中验证有效的经验与组员的新架构进行了讨论和融合。融合时，我重点保留了以下任务级经验：

- 状态驱动的 legal actions；
- handoff 和任务语义约束；
- LLM 输出前置 verifier；
- failure-specific ban；
- partial fallback；
- recovery 动作设计。

组员的新架构提供了更统一的调度和仲裁框架，而我的 Sort 实验提供了经过验证的任务级规则和失败恢复机制。融合后，Sort 的优化不再只是我个人分支上的 patch，而成为小组整体方案的一部分。在新架构下，sort 单测成功的概率可以达到 9/10，相比小组昨天实验提升 20 个百分点。

Rope 方面，我的阶段化分析也为后续组员优化提供了问题基础：Rope 不能被当作普通刚体任务处理，必须显式处理“双端同时抓取、同时放置”和 PATH/IK 约束。

这次协作让我认识到，小组项目中个人贡献不仅是“写了哪些代码”，也包括将自己的实验结论转化为可复用的架构思路，并与其他成员的方案融合。

7. 收获与反思

这次实训最大的收获是，我对 LLM 与机器人执行系统之间的关系有了更清晰的认识。

第一，baseline 非常重要。没有 baseline，就很难判断问题来自模型、parser、IK、碰撞还是任务语义。第一天的六任务评测帮助我建立了问题地图，也帮助小组决定优化优先级。

第二，LLM 不能替代任务约束。Sort 的实验说明，哪怕模型能输出格式正确的动作，也可能物理不可执行。因此需要 legal actions、verifier 和 fallback 把 LLM 约束在可执行空间内。

第三，多机器人任务必须重视失败恢复。一个机器人失败时，不应该简单放弃整轮计划，而应该尽量保留其他可执行动作，并通过 partial fallback 推进状态。可以加入反思机制，喂入下一次对话的 prompt 中，减少模型重新犯相同错误的概率。

第四，不同任务需要不同抽象。Sort 更适合离散状态和 handoff 规则；Rope 更依赖阶段协同、PATH 和 IK；Pack/Sweep/Cabinet/Sandwich 又有各自不同的语义约束。因此，通用架构必须允许任务级规则接入。

最后，我也体会到工程协作的重要性。个人分支中的成功经验如果不能与小组架构融合，就很难成为最终系统的一部分。通过第二天的架构讨论和融合，我学会了如何把个人任务级优化转化为小组可复用的方案。